

Informe Técnico: Análisis y Diseño de Resiliencia

Práctica de Tolerancia a Fallas en Microservicios

Integrante 1: Nombre Apellido

Integrante 2: Nombre Apellido

Curso de Sistemas Distribuidos
Universidad/Institución

19 de julio de 2026

Índice

1. Introducción	3
2. Análisis de Fallo 4: Base de Datos Intermitente (Conectividad)	3
2.1. Explicación Teórica y Fundamentos Distribuidos	3
2.2. Relación con Clases e Investigación MLR	3
2.3. Solución Propuesta a Nivel de Producción	3
2.4. Pseudocódigo o Diagrama de Solución	3
3. Análisis de Fallo 6: Condición de Carrera (Consistencia)	4
3.1. Explicación Teórica y Fundamentos Distribuidos	4
3.2. Relación con Clases e Investigación MLR	4
3.3. Solución Propuesta a Nivel de Producción	4
3.4. Pseudocódigo o Diagrama de Solución	4
4. Conclusiones	5
5. Referencias Bibliográficas	5

1. Introducción

Este informe técnico aborda el análisis teórico y diseño de soluciones a nivel de producción para los dos escenarios de fallo que no fueron implementados en el código práctico de la arquitectura del clúster.

Los fallos analizados bajo fundamentos de sistemas distribuidos y resiliencia en este documento son:

- **Fallo 4: Base de Datos Intermitente (Conectividad)**
 - **Fallo 6: Condición de Carrera (Consistencia)**
-

2. Análisis de Fallo 4: Base de Datos Intermitente (Conectividad)

2.1. Explicación Teórica y Fundamentos Distribuidos

Escriba aquí la explicación teórica...

2.2. Relación con Clases e Investigación MLR

Escriba aquí la relación con la literatura...

2.3. Solución Propuesta a Nivel de Producción

Escriba aquí la solución de producción propuesta...

2.4. Pseudocódigo o Diagrama de Solución

A continuación se presenta el pseudocódigo que ilustra la implementación de reintentos con backoff exponencial y jitter en la conexión de base de datos:

```
1 import time
2 import random
3
4 def conectar_con_backoff(max_intentos=5, base_delay=1.0, max_delay=30.0):
5     intento = 0
6     while intento < max_intentos:
7         try:
8             # Intentar conexión a PostgreSQL
9             db = conectar_postgres()
10            return db
11        except Exception as e:
12            intento += 1
```

```

13         if intento == max_intentos:
14             raise e
15         # Calcular delay exponencial con jitter aleatorio
16         temp = min(max_delay, base_delay * (2 ** intento))
17         sleep_time = temp / 2 + random.uniform(0, temp / 2)
18         print(f"Error de conexión. Reintentando en {sleep_time:.2f}
segundos...")
19         time.sleep(sleep_time)

```

Listing 1: Pseudocódigo de reconexión y reintentos con Jitter

3. Análisis de Fallo 6: Condición de Carrera (Consistencia)

3.1. Explicación Teórica y Fundamentos Distribuidos

Escriba aquí la explicación teórica...

3.2. Relación con Clases e Investigación MLR

Escriba aquí la relación con la literatura...

3.3. Solución Propuesta a Nivel de Producción

Escriba aquí la solución de producción propuesta...

3.4. Pseudocódigo o Diagrama de Solución

A continuación se ilustra mediante pseudocódigo la solución utilizando bloqueo pesimista a nivel de base de datos para asegurar el procesamiento atómico de la reserva:

```

1 # Flujo atomico de reserva en base de datos PostgreSQL
2 def procesar_reserva_concurrente(db_connection, asiento_id, usuario_id):
3     with db_connection.cursor() as cursor:
4         # 1. Bloquear la fila del asiento especifico (FOR UPDATE)
5         cursor.execute(
6             "SELECT disponible FROM asientos WHERE id = %s FOR UPDATE",
7             (asiento_id,)
8         )
9         row = cursor.fetchone()
10
11     if row and row[0] is True: # Si esta disponible
12         # 2. Descontar del inventario y registrar la reserva
13         cursor.execute(
14             "UPDATE asientos SET disponible = FALSE WHERE id = %s",
15             (asiento_id,)

```

```

16         )
17         cursor.execute(
18             "INSERT INTO reservas (id, asiento_id, usuario_id, estado)
VALUES (%s, %s, %s, 'CONFIRMADA')",
19             (generar_uuid(), asiento_id, usuario_id)
20         )
21         db_connection.commit()
22         return {"status": "Reserva exitosa"}
23     else:
24         db_connection.rollback()
25         return {"status": "Fallo: Asiento ya reservado"}

```

Listing 2: Pseudocódigo de control de concurrencia con Bloqueo Pesimista

4. Conclusiones

Escriba aquí las conclusiones...

5. Referencias Bibliográficas

Referencias

- [1] Fox, A., & Brewer, E. A. (1999). *Harvest, yield, and scalable tolerant systems*. Proceedings of the Seventh Workshop on Hot Topics in Operating Systems.
- [2] Referencia a tu documento MLR...
- [3] Diapositivas de clase: Tema de Tolerancia a Fallas y Concurrencia Distribuida.